UNIVERSIDADE FEDERAL DO CEARÁ • QUALIDADE DE SOFTWARE 2026.2

# Refatorações de test smells em Projetos Open Source Python


DOCÊNCIA

Profª. Dra. Carla Ilane Bezerra
Monitor Gabriel Alves Pessoa

EQUIPE

Cícero Higor (574228)
Kawan Torres (563869)

# O Projeto Selecionado: Datasette

simonw/datasette • Framework em Python

Definição do Projeto

**"Uma ferramenta open source para explorar, consultar e publicar bases relacionais SQLite através de interfaces web e APIs REST automáticas."**

Desenvolvido pelo jornalista de dados Simon Willison, o Datasette é amplamente utilizado por veículos de imprensa, cientistas de dados e empresas para transformar dados locais em serviços públicos com suporte a consultas SQL customizadas e controle de permissões.

CAMADA DE ENTREGA

### Web UI & APIs REST

Geração automática de rotas para navegação visual em tabelas e exportação instantânea de datasets em JSON e CSV.

ARQUITETURA DE PRODUÇÃO

### Servidor Assíncrono ASGI

Processamento de conexões concorrentes sobre loop de eventos, isolando transações bloqueantes do SQLite em pool de threads.

ECOSSISTEMA DE TESTES

### 70,7% Testes Assíncronos

Suíte massiva com **1.099 corrotinas async def** e 455 funções síncronas em 85 arquivos, exigindo `pytest-asyncio`.

01. DIAGNÓSTICO INICIAL

# Métricas da Linha de Base Inicial

Levantamento com TEMPY, Pytest e Radon

### TEST SMELLS (TEMPY)

**258**

Identificados em 85 arquivos de teste da suíte original.

### VOLUME DE TESTES

**2.935**

2.829 passed, 85 skipped, 6 xfail e 15 xpass. Zero falhas.

### COBERTURA GLOBAL

**90,27%**

87,40% no código de produção (`datasette/`).

### ESTRUTURA (RADON)

**4,43**

Complexidade ciclomática média das funções (MI: 43,10).

### O PARADOXO DA COBERTURA

**Alta cobertura não garante bons testes:** O Datasette tinha 90,27% de cobertura e zero falhas, mas acumulava 258 smells. As linhas de produção eram executadas, mas os testes continham asserções frouxas e desvios procedurais.

### COMPLEXIDADE EM FUNÇÕES DE TESTE

**Alerta vermelho:** Em testes, a complexidade ciclomática ideal é próxima de 1 (fluxo linear). A média de 4,43 revelou métodos contendo múltiplos desvios (ifs, loops), comprovando a existência de **dívida técnica acumulada**.

02. IDENTIFICAÇÃO

# Mapeamento dos Smells na Base e Critérios de Seleção

Classificação das 258 ocorrências segundo o TEMPY e CATSPY

| TEST SMELL DETECTADO | OCORRÊNCIAS | SEVERIDADE OFICIAL | NOSSA DECISÃO | JUSTIFICATIVA TÉCNICA DA SELEÇÃO / DESCARTE |
|---|---|---|---|---|
| **Conditional Test Logic** | 135 | ALTA | **Selecionado (24 refatorados)** | **Mais frequente na base; ramifica o teste e mascara defeitos parciais** |
| **Unknown Test** | 43 | ALTA | **Selecionado (5 refatorados)** | **Testes que capturam exceções sem checar a mensagem ou a causa** |
| **Verbose Test** | 27 | ALTA | Não selecionado | Descartado: exigiria quebrar fixtures compartilhadas complexas do core |
| **Exception Handling** | 20 | — | Não selecionado | Descartado: não consta no catálogo normativo oficial (CATSPY) |
| **Non-Functional Statement** | **10** | BAIXA | **Selecionado (7 refatorados)** | **Incluído como 5º tipo extra: elimina código inerte (pass) em context managers** |
| **Programming Paradigms Blend** | 10 | MÉDIA | Não selecionado | Descartado: padrões legítimos de orientação a objetos da arquitetura |
| **Undefined Test** | **10** | ALTA | **Selecionado (4 refatorados)** | **Funções auxiliares que continham assert fora da convenção formal** |
| **Sleepy Test** | **3*** | MÉDIA | **Selecionado (5 refatorados)** | **Elimina pausas arbitrárias que geram lentidão e falhas intermitentes** |

*Nota sobre Sleepy Test: O TEMPY analisa apenas funções síncronas (def); nós identificamos e refatoramos manualmente mais 4 ocorrências em corrotinas assíncronas (async def), totalizando 5 intervenções reais no projeto.

# Exemplo 1: Sleepy Test (Severidade Média)

Refatoração R02: tests/test_internals_database.py

**// ANTES (LINHA 674)** — test_internals_database.py

```
@pytest.mark.asyncio
async def test_execute_write_block_false(db):
    await db.execute_write(
        "update roadside_attractions set name = ? where pk = ?",
        ["Mystery!", 1],
    )
    await asyncio.sleep(0.1)  # Pausa estática arbitrária

    rows = await db.execute("select name from ... where pk = 1")
    assert "Mystery!" == rows.rows[0][0]
```

**// DEPOIS (LINHA 684)** — test_internals_database.py

```
async def wait_for_row_value(db, sql, expected, timeout=1.0):

    deadline = asyncio.get_running_loop().time() + timeout
    while asyncio.get_running_loop().time() < deadline:
        rows = await db.execute(sql)
        if rows.rows and rows.rows[0][0] == expected:
            return rows.rows[0][0]
        await asyncio.sleep(0.005)  # Polling curto de 5ms

    raise TimeoutError(f"Expected {expected!r} not found")

@pytest.mark.asyncio
async def test_execute_write_block_false(db):
    await db.execute_write("update roadside_attractions...", ...)
    name = await wait_for_row_value(db, "select name...", "Mystery!")

    assert name == "Mystery!"
```

**PROBLEMA DIAGNOSTICADO**

Pausa estática de 100ms para aguardar a thread SQLite. Se a máquina for lenta, causa **falha intermitente (flakiness)**; se for rápida, desperdiça tempo.

**SOLUÇÃO IMPLEMENTADA**

Polling assíncrono com intervalo de 5ms e timeout máximo defensivo de 1s via `wait_for_row_value`. Execução imediata e determinística.

# Exemplo 2: Undefined Test (Severidade Alta)

Refatoração R08: tests/test_queries.py

**// ANTES (LINHA 34)** — test_queries.py

```
def _template_button_sql(html, operation):
    soup = Soup(html, "html.parser")
    button = soup.select_one(f'button[data-sql-template="{operation}"]')
    # Assert em helper utilitário (fora do padrão test_*)

    assert button, f"Could not find {operation} template button"

    assert button.get("data-template-sql"), "Missing SQL"

    return button["data-template-sql"]
```

**// DEPOIS (LINHA 36)** — test_queries.py

```
def _template_button_sql(html, operation):
    soup = Soup(html, "html.parser")
    button = soup.select_one(f'button[data-sql-template="{operation}"]')
    # Validação defensiva explícita com raise ValueError

    if not button:

        raise ValueError(f"Could not find {operation} template button")

    sql = button.get("data-template-sql")
    if not sql:

        raise ValueError("Missing SQL")

    return sql
```

**PROBLEMA DIAGNOSTICADO**

Funções utilitárias sem prefixo `test_` contendo `assert` geram falsos testes não rastreados pelo framework.

**SOLUÇÃO IMPLEMENTADA**

Substituição por checagem defensiva com `raise ValueError`, mantendo a proteção defensiva do helper sem violar o padrão de teste.

# Exemplo 3: Non-Functional Statement (Severidade Baixa)

Refatoração R10: tests/test_cli_serve_server.py

**// ANTES (LINHA 127)** — test_cli_serve_server.py

```
@pytest.mark.serial
def test_startup_error_fails_fast_before_port_binds(serve_with_plugins):
    proc, port = serve_with_plugins(...)
    # ...
    # Chamada associada no with deixando corpo inerte

    with (

        pytest.raises(OSError),

        socket.create_connection(("127.0.0.1", port), timeout=0.2),

    ):

        pass
```

**// DEPOIS (LINHA 132)** — test_cli_serve_server.py

```
@pytest.mark.serial
def test_startup_error_fails_fast_before_port_binds(serve_with_plugins):
    proc, port = serve_with_plugins(...)
    # ...
    # Chamada real aninhada no corpo da verificação; pass removido

    with pytest.raises(OSError):

        socket.create_connection(("127.0.0.1", port), timeout=0.2)
```

**PROBLEMA DIAGNOSTICADO**

Instrução inerte `pass` em bloco de teste, reduzindo a clareza sobre o que de fato está sendo avaliado.

**SOLUÇÃO IMPLEMENTADA**

Remoção do `pass` e execução direta da operação conectiva dentro do context manager do Pytest.

# Exemplo 4: Unknown Test (Severidade Alta)

Refatoração R20: tests/test_template_context.py

// ANTES (LINHA 74) — test_template_context.py

```
def test_from_extra_must_match_a_registered_extra():
    from datasette.views import from_extra

    @dataclass
    class BadContext(Context):
        extras_scope = ExtraScope.TABLE
        not_a_real_extra: str = from_extra()

    # Espera apenas KeyError sem asserção sobre a mensagem!

    with pytest.raises(KeyError):

        BadContext.documented_fields()
```

// DEPOIS (LINHA 79) — test_template_context.py

```
def test_from_extra_must_match_a_registered_extra():
    from datasette.views import from_extra

    @dataclass
    class BadContext(Context):
        extras_scope = ExtraScope.TABLE
        not_a_real_extra: str = from_extra()

    # Captura com as excinfo + asserção semântica sobre a mensagem

    with pytest.raises(KeyError) as excinfo:

        BadContext.documented_fields()

    assert "no registered extra of that name" in str(excinfo.value)
```

**PROBLEMA DIAGNOSTICADO**

Ausência de asserção explícita sobre o erro capturado. Qualquer `KeyError` não relacionado passaria como **falso positivo (passa indevidamente)**.

**SOLUÇÃO IMPLEMENTADA**

Inspeção semântica com `as excinfo`, validando que a falha decorre rigorosamente do contrato da funcionalidade.

# Exemplo 5: Conditional Test Logic (Severidade Alta)

Refatoração R42: tests/test_csv.py

**// ANTES (LINHA 191)** — test_csv.py

```
@pytest.mark.asyncio
@pytest.mark.parametrize("download", (False, True))
@pytest.mark.parametrize("query_string,expected_error", (...))
async def test_custom_sql_csv_errors(ds_client, query_string, ...):
    # Desvio condicional interno que bifurca a execução do teste

    if download:

        query_string += "&_dl=1"

    response = await ds_client.get(f"/fixtures/-/query.csv?{query_string}")
    assert response.status_code == 400
    assert response.text == expected_error
```

**// DEPOIS (LINHA 191)** — test_csv.py

```
@pytest.mark.asyncio
 # Parametrização declarativa direta do sufixo da URL

 @pytest.mark.parametrize("download_suffix", ("", "&_dl=1"))

@pytest.mark.parametrize("query_string,expected_error", (...))
async def test_custom_sql_csv_errors(ds_client, query_string, expected_erro
     response = await ds_client.get(f"/fixtures/-/query.csv?{query_string}{

    assert response.status_code == 400
    assert response.text == expected_error
```

**PROBLEMA DIAGNOSTICADO**

Uso de `if download:` dentro do teste, criando dois caminhos de execução e elevando a Complexidade Ciclomática (CC = 6, Rank B).

**SOLUÇÃO IMPLEMENTADA**

Parametrização declarativa eliminando o branch (CC caiu de 6 para 5). *Nota: a CC residual de 5 decorre dos 4 asserts contados pelo Radon + 1 base da função.*

# Experiência com o CATSPY e Oportunidades

Desafios práticos na consulta e aplicação do catálogo

01

## Foco em Código Síncrono (Dificuldade com Async/Await)

O catálogo prescreve receitas pensadas em testes síncronos tradicionais (como `time.sleep`). Em arquiteturas assíncronas com `pytest-asyncio`, não basta substituir o sleep por esperas convencionais; foi necessário conceber polling não bloqueante no event loop e sincronização determinística via `asyncio.Event`.

02

## Exemplos Didáticos vs. Código Real de Produção

Os exemplos do catálogo ilustram funções isoladas de 3 a 4 linhas em cenários ideais. Em um software real, os testes dependem de fixtures compartilhadas, injeção de dependência e mocks interdependentes. Falta orientação no catálogo sobre como refatorar sem desestruturar a arquitetura de suporte dos testes.

03

## Ausência de Análise de Trade-offs e Riscos

O catálogo instrui como eliminar o smell, mas não discute o custo-benefício. Por exemplo, quebrar certas fixtures do core para sanar Verbose Test ou parametrizar laços gigantescos pode inflar o tempo de execução e elevar o risco de regressão. O catálogo deveria ponderar quando *não* refatorar.

05. RESULTADOS

# Estado Final: Tabela Comparativa Geral

Medições comparativas: Linha de Base versus Estado Final

**SMELLS ELIMINADOS**
**-50**
Queda de 258 para 208 ocorrências (-19,38%).

**TESTES APROVADOS**
**2.850**
+21 testes (2.829 ➔ 2.850 passed por parametrização).

**FALHAS / REGRESSÕES**
**0**
100% de estabilidade preservada na aplicação.

**COMPLEXIDADE MÉDIA**
**4,38**
Redução de 4,43 para 4,38 (-1,13% no Radon).

| MÉTRICA AVALIADA | ANTES (LINHA DE BASE) | DEPOIS (ESTADO FINAL) | VARIAÇÃO | AVALIAÇÃO DE QUALIDADE |
|---|---|---|---|---|
| **Test Smells Totais (TEMPY)** | 258 | 208 | **-50 (-19,38%)** | **Meta superada em 25% (foco em Severidade Alta)** |
| **Testes Aprovados (Passed)** | 2.829 | 2.850 | +21 (+0,74%) | Ganhos derivados da decomposição declarativa de laços |
| **Cobertura Total do Projeto** | 90,27% | 90,27% | 0,00 p.p. | Preservação rigorosa da cobertura de código global |
| **Cobertura Código Produção (datasette/)** | 87,40% | 87,43% | +0,03 p.p. | Leve incremento na aplicação de produção |
| **Complexidade Ciclomática Média** | 4,43 | 4,38 | -0,05 (-1,13%) | **Linearização efetiva dos métodos de teste refatorados** |
| **Índice de Manutenibilidade Médio (MI)** | 43,10 | 43,09 | -0,01 | Padrão excelente de manutenibilidade mantido (>40) |

Reconciliação: Refatorações em CTL removeram múltiplos branches e nós da AST simultaneamente (ex: R24 removeu 6 pontos de uma vez), totalizando 50 reduções apontadas pelo TEMPY nas 45 intervenções realizadas.

# Contribuição no Fork e Lições Aprendidas

Pull Request #1 • cicerocarnauba/datasette

## Evidência do Trabalho no GitHub

Todas as intervenções foram consolidadas no **Pull Request #1** aberto na branch `refatoracao-test-smells` do nosso fork oficial.

- **48 commits atômicos** e documentados com histórico limpo;
- **19 arquivos de teste** modificados estritamente dentro de `tests/`;
- **Ambos superamos a cota:** 23 refatorações feitas por Kawan e 22 por Higor;
- **0 falhas e 0 erros** na suíte completa de testes.

## Nossas Lições Aprendidas

A dívida técnica diagnosticada na base foi paga com sucesso. A refatoração de testes não é cosmética: torna a suíte rápida, determinística e com diagnóstico cirúrgico de regressões na integração contínua.

> "Qualidade de teste significa código linear, atômico e determinístico — e não apenas uma porcentagem alta de cobertura."




# Muito Obrigado pela Atenção!

Refatorações de test smells em Projetos Open Source Python

Cícero Higor • Kawan Torres
Profª. Dra. Carla Ilane Bezerra • Monitor Gabriel Alves Pessoa